

PQC 서명 크기가 B+tree 구조에 미치는 영향 분석

SQLite DB에서의 오버플로우 및 팬아웃 병목 현상과 저장 전략

발표자: 이승원

Contents

1 연구 배경 및 문제 정의

2 B+tree 구조에서의 핵심 병목

3 저장 전략 및 실험 설계

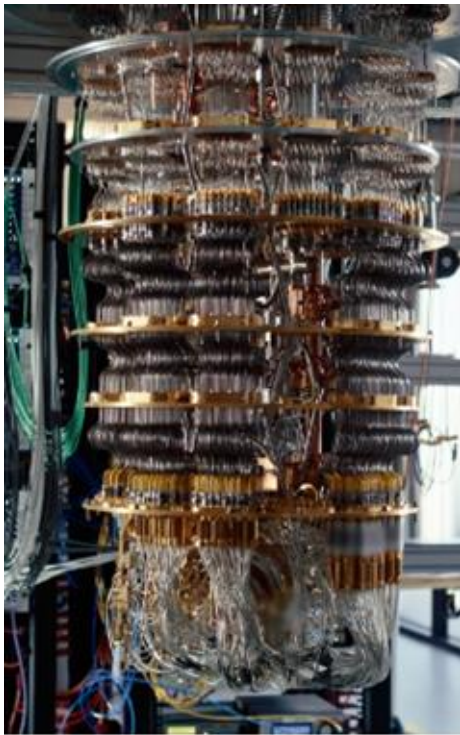
4 핵심 결과 분석

5 결론

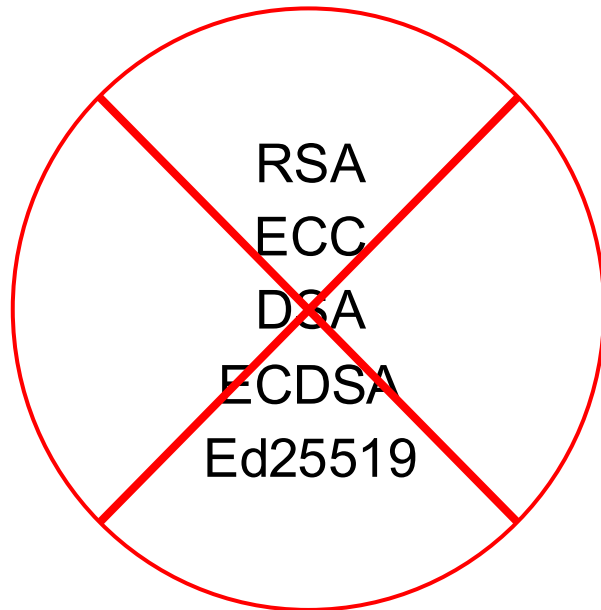


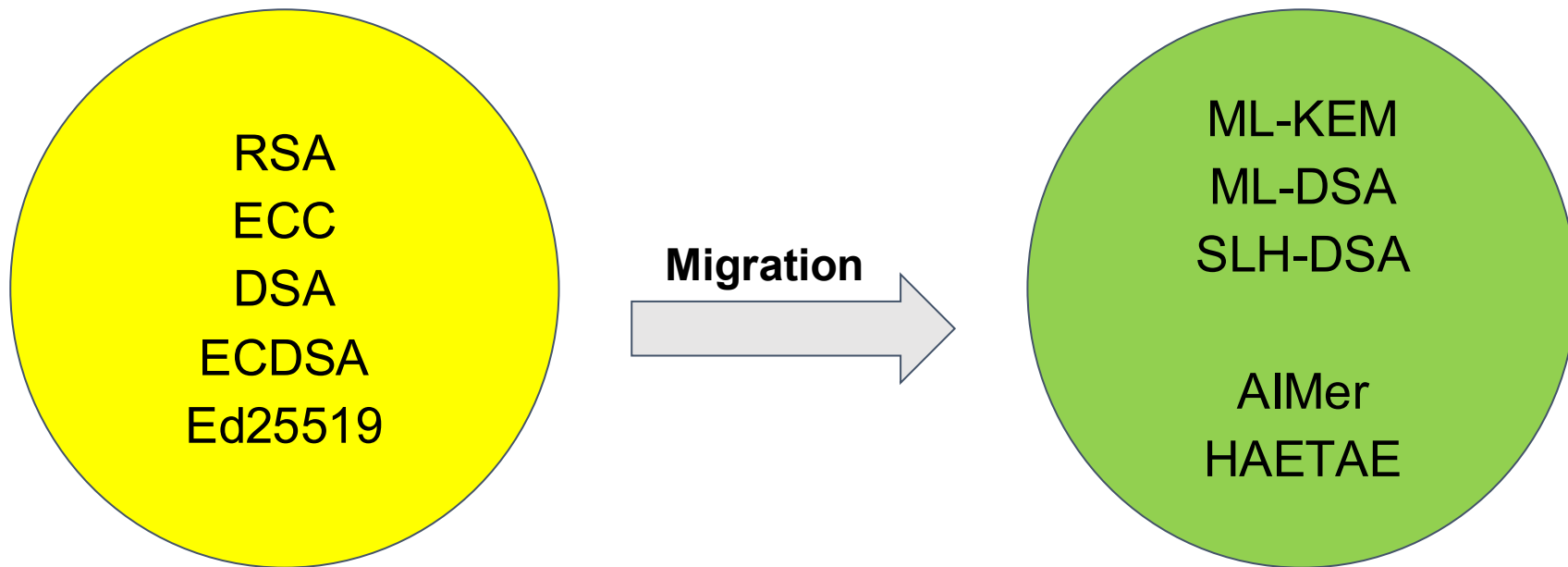
1

연구배경



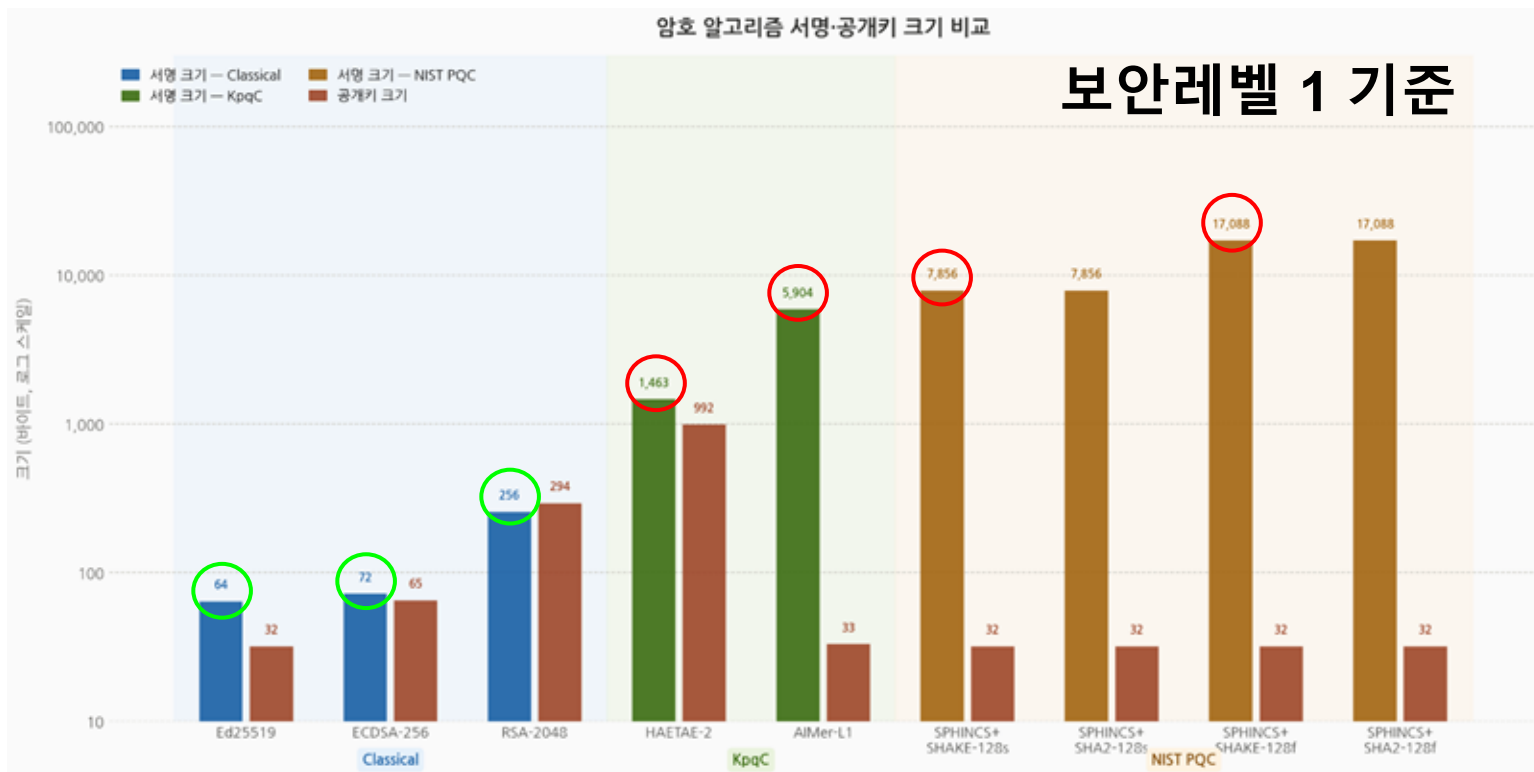
Shor
Grover





1

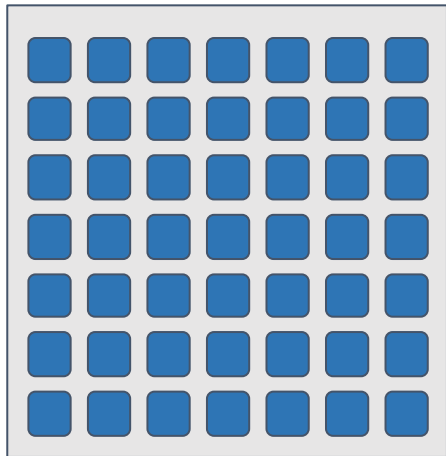
문제 정의



1

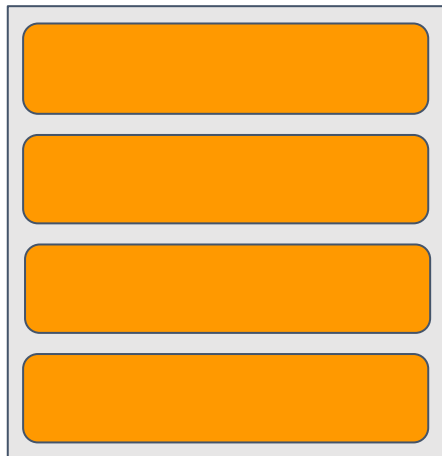
문제 정의

Classical Algorithm

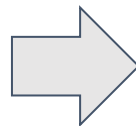


페이지 내 저장 가능

PQC Algorithm



페이지 여유 감소



DB 성능 저하

2

B+tree 구조에서의 핵심 병목

큰 서명 → 페이지 초과 → 오버플로우 및 fan-out 감소 → 삽입/조회 성능 저하

임계 값 초과

6kB
전자 서명

4kB
페이지

4kB

오버플로우 페이지

2
k
B

4kB

오버플로우 페이지

2
k
B

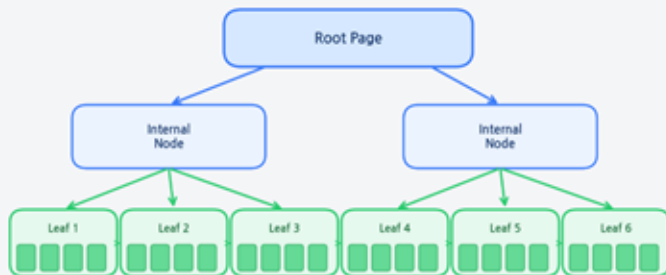
4kB

랜덤 I/O
증가

2

B+tree 구조에서의 핵심 병목

정상: 레코드가 작을 때



페이지당 레코드 4개

→ 리프 페이지 6개로 충분 | 트리 낮음 | 탐색 빠름

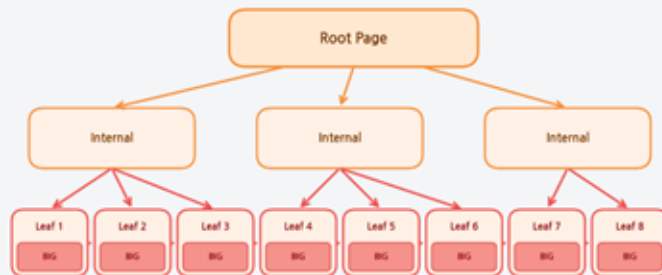
레코드 크기

648

→ 작아서 한 페이지에 여러 개 들어감

리프 페이지 수 적음 → 성능 우수

문제: 레코드가 클 때 (PQC 서명)



페이지당 레코드 1개 (너무 커서)

→ 리프 페이지 8개 필요 | 트리 높아짐 | 탐색 느림

레코드 크기

17,088

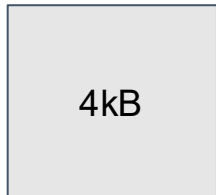
→ 너무 커서
1개도 백백

리프 페이지 수 증가 → 범위 조회 비용 폭증

3

저장 전략

A전략

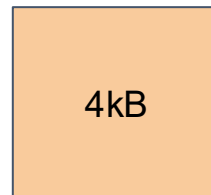


단일 테이블 저장
SQLite 기본 값
Baseline

C전략



Meta



BLOB

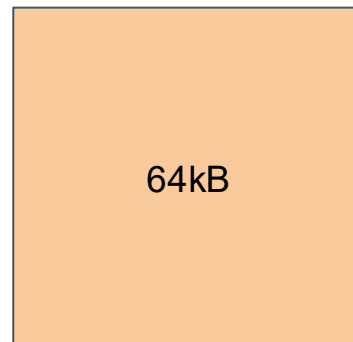
수직 파티셔닝 저장
Meta=메타 데이터
BLOB=서명 본문
SQLite 기본 값

B전략



A전략에서 페이지 크기만 변화

D전략



C전략에서 BLOB페이지
크기만 변화
C전략의 대조군

3

실험 설계

실험환경

Apple M2
8GB RAM
250GB SSD
SQLite 3.51.0
Python 3.14.2

Ed25519

RSA-2048

ECDSA-256

AlMer-L1

HAETA-E-2

SPIHNCS+-SHAKE-128 s/f

SPIHNCS+-SHA2-128 s/f

전략 A

전략 B

전략 C

전략 D

평가 지표

블록 삽입 처리량(100만 건)
범위 조회 지연시간(10만 건)
오버플로우 페이지 수
트리 깊이
저장 공간 사용량

암호 연산 시간을 배제하기 위해 공개키, 서명, 메시지 해시 미리 생성 + 10,000개의 서명 풀을 순환 재사용

4

핵심 실험 결과 분석

전략 A	삽입 처리량	범위 조회 지연(ms)	오버플로우 페이지 수	트리 깊이
Ed25519 (64B)	14,173	123.066	0	3
ECDSA-256 (72B)	12,459	220.296	0	3
RSA-2048 (256B)	12,459	270.201	0	3

전략 A	삽입 처리량	범위 조회 지연(ms)	오버플로우 페이지 수	트리 깊이
AlMer-L1 (5,904B)	5,981	9812.3	1,000,000	4
HAETAE-2 (1,463B)	12,107	3832.482	0	4
SPHINCS+-128s (7,856B)	7,684	10076.612	1,000,000	4
SPHINCS+-128f (17,088B)	5,638	19320.947	4,000,000	4

4

핵심 실험 결과 분석

삽입 처리량 (rps)



AIMer: 3.9배 향상

HAETAE-2: 2.3배 향상

SPHINCS+: 2.9배 향상

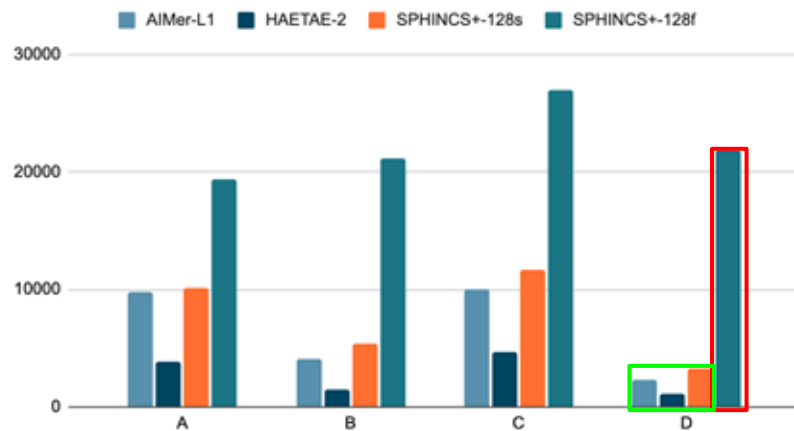
전략 B에서 가장 개선 효과가 큼

→ 페이지 크기를 늘린 결과로 오버플로우가 제거 되면서 삽입 성능 개선

4

핵심 실험 결과 분석

범위 조회 지연시간 (ms)



전략 D에서 가장 개선 효과가 큼
하지만 초대형 서명은 회복 실패

→ 범위 조회는 오버플로우 문제가 아니라 fan-out 감소로 리프페이지 수가 늘어나고 순차 읽기 비용이 커지는 구조적 병목 발생

4

핵심 실험 결과 분석

Table+Overflow 합산	전략 A	전략 B	전략 C	전략 D
AlMer-L1	5,864MB	6,252MB	5,955MB	6,348MB
HAETA-E-2	3,916MB	2,605MB	4,007MB	2,594MB
SPHINCS+-128s	7,822MB	7,815MB	7,913MB	7,913MB
SPHINCS+-128f	16,604MB	31,260MB	16,695MB	20,949MB

→ 전략 B는 개선 효과가 크지만 페이지 크기로 인해 저장 공간이 커짐

→ 전략 D는 성능과 공간 효율의 균형점

소형 서명

HAETAE-2는 모든 전략에서 오버플로우가 발생하지 않아 삽입 처리량을 우선시 할 때는 전략 B, 저장 공간과 범위 조회를 동시에 고려할 경우 전략 D가 적합하다.

중형 서명

AIMer-L1, SPHINCS+-128s는 오버플로우가 병목이므로 전략 B에서 가장 큰 개선효과를 보이며, 저장 공간을 고려할 경우엔 전략 D가 대안이다.

대형 서명

SPHINCS+-128f은 서명 크기가 너무 커 어떤 전략으로도 회복이 불가능한 구조적 한계가 존재한다.

→ 전략 C는 테이블 분리를 해도 테이블 크기가 작기 때문에 오버헤드만 가중되어 권장하지 않는다.

Thank you

